

An Optimized FPGA Arithmetic Unit Using Dadda Multiplier and Squaring Unit

M Adithya Sai, Shaik Sharfuddin, V Sai Santhosh, P. Ashok

Department of Electronics and Communication Engineering

Amrita School of Engineering, Coimbatore

Amrita Vishwa Vidyapeetham, India

p_ashok@cb.amrita.edu

Abstract—Since multiplication and squaring operations are dominant in the VLSI/FPGA implementation of signal processing systems, it is important to ensure that the arithmetic units are able to offer high speed operation with low power consumption. In this paper, we proposed a reconfigurable Dadda based arithmetic unit that can support the execution of multiplication and squaring operations in a unified manner. In addition, the proposed architecture includes the utilization of a multiplexer based datapath, which can switch operation based on the operands, ensuring that there is no redundant hardware utilization in the execution of squaring operations. In order to further minimize hardware, delay, and switching activities in non-critical reduction paths, the proposed architecture utilizes an approximate 4:2 compressor, along with data reordering to minimize switching activities, thus ensuring low dynamic power dissipation. The proposed architecture has been implemented on an RTL model, which has been mapped to an FPGA platform to validate its functionality. Experimental results have confirmed that the proposed architecture outperforms the conventional Dadda based architecture in terms of hardware utilization, power dissipation, and critical path delay, making it suitable for high-speed arithmetic operation. In addition, the proposed architecture can be reused to support other hardware applications, thus making it more suitable for implementation in various signal processing systems.

Index Terms—Reconfigurable arithmetic unit, Dadda multiplier, squarer, approximate 4:2 compressor, input reordering, low power, high speed, FPGA implementation

I. INTRODUCTION

Arithmetic operations like addition, multiplication, and squaring are at the core of embedded systems, image processing, signal processing, and communications. The design of arithmetic operations directly affects the silicon area and power consumption in VLSI and FPGA implementations. With the increase in clock speed and system complexity, designers face the challenge of implementing arithmetic operations that are fast and consume less power.

Because multiplication generates and gathers partial products, it is one of the most computationally demanding operations. As a result, multiplier architecture is crucial for figuring out system performance and energy efficiency. A number of multiplier architectures, including array, Booth, Wallace, and Dadda multipliers, have been proposed to address the trade-off between speed, area, and power consumption. Each architecture offers advantages based on the requirements of the application.

Dadda multipliers are widely used in high-speed arithmetic circuits due to their superior partial product reduction technique, which minimizes the number of reduction stages when compared to Wallace multipliers. Performance is enhanced and the critical path delay is decreased as a result. However, because of the high switching activity in the reduction tree, conventional Dadda multiplier implementations still have high power consumption and routing complexity. Additionally, when both multiplication and squaring are required in the same system, the logic is duplicated and the resources are used inefficiently because squaring operations are frequently performed using specialized hardware blocks.

In order to address these issues, recent research has focused on low-power and approximate computing techniques that trade limited computational accuracy for significant reductions in power consumption, latency, and hardware complexity. Switching activity reduction techniques and approximate compressors have demonstrated promising results, particularly in applications where errors are tolerated. Nevertheless, most existing designs are based on fixed-function architectures that optimize multipliers and squarers independently and offer minimal support for reconfigurable operation within a single datapath.

Based on the architecture of a Dadda multiplier and squarer, this study proposes a reconfigurable arithmetic unit that is fast and energy-efficient. The design employs a comparator-controlled, multiplexer-based datapath to reduce unnecessary hardware by dynamically selecting between multiplication and squaring modes depending on input conditions. Input reordering and roughly 4:2 compressor techniques are also used to reduce switching activity, critical path delay, and hardware overhead. In order to demonstrate improvements in speed, power consumption, and space usage, the proposed design is constructed and tested on an FPGA platform and its performance is contrasted with that of a standard Dadda multiplier.

II. LITERATURE SURVEY

Due to growing performance and power limitations in contemporary FPGA and VLSI based systems, energy efficient arithmetic unit design has received a lot of attention, especially for image processing, signal processing, and error tolerant

computing applications. By permitting controlled computational inaccuracies, approximate computing techniques have been extensively investigated to lower power consumption, delay, and hardware complexity.

In order to lower power consumption and the power–delay product, Chakraborty *et al.* [1] An approximate multiplier purposed for image processing applications. Despite the design’s significant energy savings, accuracy degradation under stringent image quality constraints is still an issue.

Da Rosa *et al.* [2] introduced AxRSU- 2^m , a class of higher-order approximate encoders for radix- 2^m squarer units, demonstrating improved hardware efficiency and reduced power consumption. The work focuses solely on squarer architectures and does not support reconfigurable arithmetic operations.

A high-speed and energy-efficient robust 4:2 compressor, emphasizing delay reduction and fault tolerance was developed by Rajput *et al.* [3]. While developed at the compressor level, the design does not explore approximation-based power savings or multi-function arithmetic units.

Rashidi [4] proposed low-cost approximate multipliers for image processing by simplifying compressor designs, resulting in reduced area and power consumption. The maintenance of accuracy at an appropriate level in different applications remains an issue of concern.

Krishna *et al.* [5] presented a probability-based approximate 4:2 compressor, focusing on reducing error rates as well as power consumption. The design is appropriate for error-tolerant applications but does not consider reducing switching activity or architectural reconfigurability.

Kavand *et al.* [6] investigated exact and approximate 4:2 compressors, multipliers using RFET technology, focusing on energy efficiency benefits at the device level. The design is technology-dependent, but the issue of reducing switching activity or architectural reconfigurability is not considered.

Yan *et al.* [7] proposed an efficient design of an approximate floating-point FFT processor, focusing on significant benefits in terms of area and power consumption. The design targets system-level FFT architectures, not arithmetic unit design.

Sayadi *et al.* [8] presented two designs of an approximate unsigned multiplier using novel 4:2 compressors, focusing on reducing delay as well as power consumption. The design is fixed-function, not flexible enough to implement different arithmetic operations.

Chen *et al.* [9] investigation on exact and approximate squarer architectures for error tolerant applications, focusing trade-offs between accuracy and hardware efficiency. The study is limited to squarer units without integration into reconfigurable designs.

Zacharelos *et al.* [10] presented approximate squaring circuits exploiting recursive architectures, achieving improved scalability and reduced power consumption. However, recursive structures increase design complexity and do not support combined arithmetic operations.

Da Rosa *et al.* [11] proposed an energy-efficient VLSI squarer unit using optimized radix- 2^m logic, demonstrating

reductions in power and area. The architecture is optimized exclusively for squaring operations.

Bodapati *et al.* [12] An low-power, high-speed scalable approximate multiplier, focusing on performance improvement through approximation. Input reordering and switching activity minimization were not addressed.

Hasan *et al.* [13] introduced a high-speed XOR–XNOR-based hybrid full adder optimized for delay reduction. While beneficial at the adder level, the work does not extend to multiplier or squarer architectures.

Raj *et al.* [14] A high-speed 8T full adder design was presented by achieving improved speed and power efficiency. The study is limited to basic arithmetic blocks without system-level integration.

Reddy *et al.* [15] proposed an approximate Booth squarer for error-tolerant computing, demonstrating reduced area and power consumption. The design is dedicated to squaring and lacks reconfigurable capability.

The accuracy of stochastic computing-based arithmetic and combinational circuits, highlighting error characteristics and reliability challenges was analysed by Ashok and Sundari [16]. Although stochastic computing offers fault tolerance, it is not directly applicable to conventional deterministic multiplier architectures.

A pipelined low-power MAC unit with sparsity-aware encoding for neural network applications, achieving energy efficiency through data-level optimization was investigated by Ramesh [17]. The work targets MAC architectures rather than standalone reconfigurable arithmetic units.

Harish *et al.* [18] presented a low-power, high-accuracy approximate multiplier for error-resilient image processing, demonstrating improved accuracy compared to conventional approximate designs. The architecture remains fixed-function and does not support squaring operations.

An FPGA-based CNN accelerator using an extended exact Booth multiplier, focusing on high-performance convolution operations was developed by JK and Prabhu [19]. The design prioritizes accuracy and throughput, with limited emphasis on power reduction through approximation.

From the surveyed literature, it is evident that while approximate multipliers and squarers significantly reduce power consumption and hardware complexity, most existing designs are fixed-function and focus on either multiplication or squaring. Furthermore, limited efforts have been made to combine input reordering and approximate compression techniques within a Dadda-based framework or to develop reconfigurable architectures that efficiently support both operations. These gaps motivate the proposed reconfigurable, high-speed, and energy-efficient arithmetic unit. Based on the comprehensive review of existing literature, it is clear that there remains a strong need for a unified and reconfigurable arithmetic architecture that simultaneously addresses power efficiency, speed, and hardware utilization. The following section presents the proposed methodology, detailing the architectural design choices and low-power optimization techniques employed to address the identified research gaps.

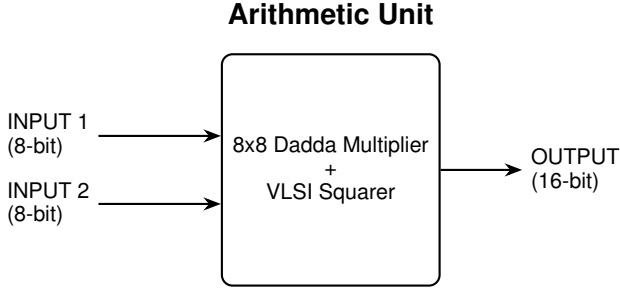


Fig. 1: Architecture of the Arithmetic Unit.

The proposed work focuses on the design and implementation of a high-speed and energy-efficient reconfigurable arithmetic unit capable of performing both multiplication and squaring operations. The methodology follows a structured design flow consisting of architectural selection, datapath optimization, low-power technique integration, and FPGA-based validation.

A. Problem Definition

Conventional Dadda multiplier and squarer architectures suffer from high power consumption and increased delay due to excessive switching activity and complex partial product reduction networks. When multiplication and squaring operations are implemented as separate hardware blocks, redundant logic and inefficient resource utilization further degrade overall system performance. Therefore, there is a need for a unified arithmetic architecture that improves speed and power efficiency while minimizing hardware overhead.

B. Proposed Architecture Overview

To address these challenges, a reconfigurable arithmetic unit based on a Dadda multiplier is proposed. The architecture employs a comparator-controlled, multiplexer-based datapath that dynamically selects the operational mode based on input conditions. When the input operands are equal, the circuit operates in squarer mode otherwise, it functions as a multiplier. This design helps in achieving efficiencies by restructuring its function to avoid extra computations and better utilize the resources.

C. High-Speed Implementation

It operates at high speed as its datapath is optimized to reduce the number of signal paths and minimize switching bottlenecks. The design of the generation and reduction phases of the Dadda multiplier has been done in an efficient manner to reduce the critical path delay. The datapath controlled by multiplexers ensures that only the required paths are enabled during computation.

D. Low-Power Optimization Techniques

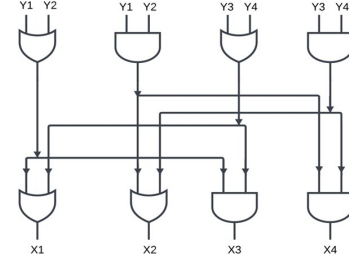


Fig. 2: Circuit Diagram of Input Reordering Circuit

In the proposed low-power architecture, an input reordering circuit is integrated into the 4:2 compressor stage to reduce switching activity during partial product reduction. The primary objective of this circuit is to minimize unnecessary logic transitions, which are a major contributor to dynamic power consumption in arithmetic units.

The reordering mechanism rearranges the input bits such that logic '1's are shifted toward the most significant bit (MSB) positions and logic '0's toward the least significant bit (LSB) positions. As a result, different input combinations containing the same number of logic '1's produce identical reordered outputs. This ensures consistent input patterns at the compressor stage, independent of the original bit positions.

For example, the input patterns 1000, 0100, 0010, and 0001 are all reordered to 1000, resulting in the same compressor output. Consequently, the total number of distinct input combinations is reduced from 16 to 6. This reduction significantly lowers switching activity within the compressor logic, leading to decreased dynamic power consumption without affecting the final arithmetic result.

Overall, the input reordering circuit improves energy efficiency by simplifying input patterns, reducing logic complexity, and minimizing redundant transitions, making it well-suited for low-power FPGA and VLSI arithmetic implementations.

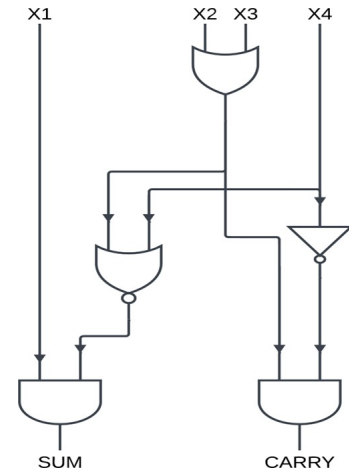


Fig. 3: Circuit Diagram of Approximate 4:2 Compressor

To reduce power consumption, minimize critical-path delay, and reduce hardware complexity, the design incorporates an approximate 4:2 compressor in the arithmetic unit. Unlike a precise compressor, which provides precise sum and carry with complex, intricate logic, the approximate compressor makes a small sacrifice in precision for less critical components of the calculation to achieve power efficiency.

An approximate 4:2 compressor has four inputs X_1 , X_2 , X_3 , and X_4 which produce two outputs, Sum and $Carry$. It uses Boolean expressions to calculate Sum and Carry, which simplifies the design, reducing the number of gates and signal chatter within the compressor.

The output equations of the approximate 4:2 compressor are given as:

$$Sum = X_1 \cdot \overline{(X_2 + X_3 + X_4)} \quad (1)$$

$$Carry = (X_2 + X_3) \cdot \overline{X_4} \quad (2)$$

Due to the use of simplified logic, a small approximation error may be introduced in the less significant output bits. However, this error has a negligible impact on overall arithmetic accuracy while providing significant reductions in power consumption, delay, and area. As a result, the approximate 4:2 compressor is well suited for high-speed and low-power arithmetic applications.

E. FPGA Implementation and Evaluation

The proposed arithmetic unit is described using RTL-level hardware description language and synthesized for FPGA implementation. Functional verification is performed through simulation, followed by hardware validation on an FPGA platform. Performance metrics such as critical path delay, power consumption, and area utilization are extracted and compared with those of a conventional Dadda multiplier to evaluate the effectiveness of the proposed methodology.

IV. EXPERIMENTAL RESULTS

The proposed reconfigurable arithmetic unit was implemented and evaluated to verify its functional correctness and performance improvements over a conventional Dadda multiplier. The design was described at the RTL level and synthesized for FPGA implementation. Functional verification was first carried out through simulation, followed by hardware validation on the FPGA platform.

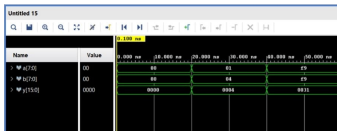


Fig. 4: Simulation Results of Proposed Arithmetic Unit

Fig.4 shows the simulation waveform of the proposed reconfigurable arithmetic unit. The output Y correctly follows the applied 8-bit input operands A and B for different test

cases, validating both multiplication and squaring operations. The observed output transitions match the expected arithmetic results at each input change, confirming the functional correctness of the RTL implementation. These results verify correct operation prior to FPGA hardware validation.

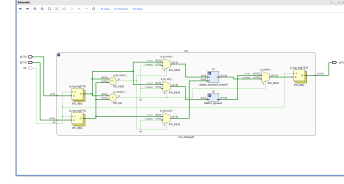


Fig. 5: RTL Schematic of Proposed Dadda Multiplier

Fig.5 illustrates the RTL schematic of the proposed reconfigurable arithmetic unit. A comparator is used to detect the equality of the 8-bit input operands, and the generated control signals drive the multiplexers to select either the Dadda multiplier or the squarer datapath. When the inputs are equal, the squarer module is activated; otherwise, the multiplier module is selected. The final output is routed through a multiplexer to produce the correct 16-bit result while avoiding redundant computations.

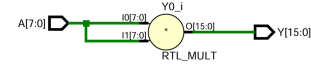


Fig. 6: RTL Schematic of Squarer Implementation

Fig.6 shows the RTL schematic of the squarer unit. The 8-bit input A is provided to both inputs of the multiplier, and hence the block calculates the product of A with itself. The 16 bit output Y of the block essentially calculates the square of the input A .

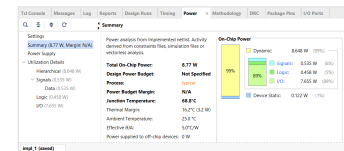


Fig. 7: Power Report of Proposed Dadda Multiplier

Fig.7 provides a post-synthesis view of the power consumption of the proposed arithmetic unit. The on-chip power dissipation occurs mainly due to dynamic power dissipation. The dynamic power dissipation occurs due to I/O switching activity and switching of internal logic. The reduced power dissipation in logic and signal levels indicates that switching activity has been reduced efficiently due to input reordering and approximate compression techniques.

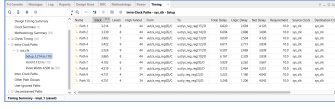


Fig. 8: Setup Time Analysis of Proposed Dadda Multiplier

Fig.8 shows the post-implementation timing setup analysis result of the proposed arithmetic unit. With a clock constraint of 10 ns, the design has a worst-case setup slack of 3.214 ns with a critical path delay of 6.820 ns. The positive slack value indicates that the design satisfies the timing requirements, with the data signals arriving at the correct time before the active edge of the clock. The result indicates that the proposed architectural optimizations are safe and can support high-speed operation at a maximum frequency of about 146 MHz.

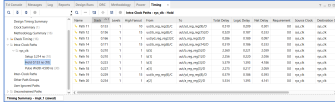


Fig. 9: Hold Time Analysis of Proposed Dadda Multiplier

Fig.9 illustrates the post-implementation analysis of the hold timing for the proposed arithmetic unit. The worst-case hold slack in the system clock domain is 0.133 ns. Since the hold slack is positive, there are no violations in the hold-time constraints. This analysis verifies that the data signals remain stable for the required hold time after the active edge of the clock, ensuring correct synchronous behavior. The positive hold margin also verifies that the optimized datapath and reconfigurable architecture do not introduce early transitions in the data signals.

TABLE I: Overall Performance Comparison of Base and Proposed Designs

Metric	Base Design (A. Chakraborty et al. [1])	Proposed Design
Energy Efficiency	Poor (10.561 W)	Good (8.77 W)
Delay (Speed)	Moderate(Hold : 2.304 ns , Setup : 5.515 ns)	Moderate(Hold : 2.260 ns , Setup : 5.222 ns)
Area Utilization	Moderate(71 LUTs, 32 IOBs)	Moderate(99 LUTs, 32 IOBs)
Suitability	Moderate - Accuracy systems like - image processing applications.	Approximate DSP applications with correlated inputs - High Accuracy if the inputs are same.

Table.1 shows the comparison between the original base design and the proposed arithmetic unit design. In this new design, we have reduced the total power from 10.561 W to 8.77 W, mainly due to switching activity reduction, with timing performance comparable to or even slightly better than that of the original, with a faster setup time but almost identical hold time. There is a moderate increase in LUTs, making it a better fit for approximate computing in DSP applications, especially with correlation in input operands.

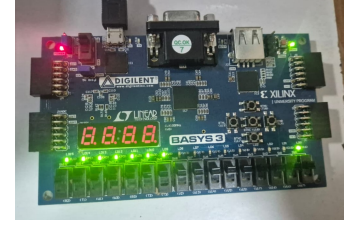


Fig. 10: Hardware Output of the Proposed Arithmetic Unit Implemented on FPGA

Fig.8 shows the hardware realization of the proposed design for the reconfigurable arithmetic unit using the Basys 3 FPGA development board. The design has been successfully implemented on the Basys 3 FPGA development board, with real-time inputs being provided using the switches on the board. The output has also been observed using the LEDs, which shows that the design is working correctly for both multiplication and squaring operations. The hardware results match the simulation results, which confirm that the design is correct.

REFERENCES

- [1] Chakraborty, Adrija, Vishal Pranao Amarnath Kumar, Akash Kumar Vruddhula, Jagannadha Naidu, and Balamurugan S. "Energy efficient approximate multiplier for image processing applications." *Results in Engineering* 25 (2025): 103798.
- [2] da Rosa, Morgana M. A., Guilherme Paim, Eduardo A. C. da Costa, Rafael Soares, and Sergio Bampi. "AxRSU-2m: Higher-Order m-Bit Approximate Encoders for Radix-2m Squarer Units." *Circuits, Systems, and Signal Processing* 43, no. 6 (2024): 3649–3678.
- [3] Rajput, Anju, Renu Kumawat, Jyoti Sharma, and Avireni Srinivasulu. "Design of Novel High Speed Energy Efficient Robust 4:2 Compressor." *Journal of VLSI Circuits and Systems* 6, no. 2 (2024): 53–64.
- [4] Rashidi, Bahram. "Efficient and low-cost approximate multipliers for image processing applications." *Integration* 94 (2024): 102084.
- [5] Krishna, L. Hemanth, Ayesha Sk, J. Bhaskara Rao, Sreehari Veeramachaneni, and Noor Mohammad Sk. "Energy-efficient approximate multiplier design with lesser error rate using the probability-based approximate 4:2 compressor." *IEEE Embedded Systems Letters* 16, no. 2 (2023): 134–137.
- [6] Kavand, Nima, Armin Darjani, Shubham Rai, and Akash Kumar. "Design of energy-efficient RFET-based exact and approximate 4:2 compressors and multipliers." *IEEE Transactions on Circuits and Systems II: Express Briefs* 70, no. 9 (2023): 3644–3648.
- [7] Yan, Chenggang, Xuan Zhao, Tingting Zhang, Jipeng Ge, Chenghua Wang, and Weiqiang Liu. "Design of high hardware efficiency approximate floating-point FFT processor." *IEEE Transactions on Circuits and Systems I: Regular Papers* 70, no. 11 (2023): 4283–4294.
- [8] Sayadi, Ladan, Somayeh Timarchi, and Akbar Sheikh-Akbari. "Two efficient approximate unsigned multipliers by developing new configuration for approximate 4:2 compressors." *IEEE Transactions on Circuits and Systems I: Regular Papers* 70, no. 4 (2023): 1649–1659.
- [9] Chen, Ke, Chenyu Xu, Haroon Waris, Weiqiang Liu, Paolo Montuschi, and Fabrizio Lombardi. "Exact and approximate squarers for error-tolerant applications." *IEEE Transactions on Computers* 72, no. 7 (2022): 2120–2126.
- [10] Zacharelou, Efstratios, Italo Nunziata, Gerardo Saggese, Antonio G. M. Strollo, and Ettore Napoli. "Approximate squaring circuits exploiting recursive architectures." *Integration* 91 (2023): 35–42.
- [11] da Rosa, Morgana M. A., Eduardo A. C. da Costa, Leandro Giacomini Rocha, Guilherme Paim, and Sergio Bampi. "Energy-Efficient VLSI Squarer Unit with Optimized Radix-2m Multiplication Logic." *Circuits, Systems, and Signal Processing* 42, no. 2 (2023): 828–852.
- [12] Bodapati, Jagadeesh, Oggi Sudahkar, and Atava G. V. Karthik Raju. "An improved design of low-power high-speed accuracy scalable approximate multiplier." *Journal of VLSI Circuits and Systems* 4, no. 1 (2022): 7–11.

- [13] Hasan, Mehedi, Md Shahbaz Hussain, Mainul Hossain, Mohd Hasan, Hasan U. Zaman, and Sharnali Islam. "A high-speed and scalable XOR-XNOR-based hybrid full adder design." *Computers & Electrical Engineering* 93 (2021): 107200.
- [14] Raj, Komma Yasaswi, Gundu Sai Vipul, Gottipati Ravindra, Rudru Gopi Venkata Krishna, and Sadulla Shaik. "Design and Performance Analysis of High Speed 8-T Full Adder." *Journal of VLSI Circuits and Systems* 3, no. 2 (2021): 1–10.
- [15] Reddy, K. Manikantta, Moodabettu Harishchandra Vasanth, Y. B. Nithin Kumar, and Devesh Dwivedi. "Design of approximate booth squarer for error-tolerant computing." *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 28, no. 5 (2020): 1230–1241.
- [16] Ashok, P., and B. Bala Tripura Sundari. "Accuracy analysis on design of stochastic computing in arithmetic components and combinational circuit." *Computation* 11, no. 12 (2023): 237.
- [17] Sriram, E. Mruthyunjai, and S. R. Ramesh. "Pipelined and Low-Power MAC Unit with Sparsity-Aware Encoding for Neural Network Applications." In *2025 IEEE 14th International Conference on Communication Systems and Network Technologies (CSNT)*, pp. 1210–1215. IEEE, 2025.
- [18] Harissh, G., R. Mathumitha, R. S. Dharini, P. A. Valli, and Navya Mohan. "Low power, high accuracy approximate multiplier for error-resilient image processing application." In *2024 IEEE International Conference on Distributed Computing, VLSI, Electrical Circuits and Robotics (DISCOVER)*, pp. 159–163. IEEE, 2024..
- [19] JK, Mona Dhakshaya, and E. Prabhu. "FPGA Based CNN Accelerator Using Extended Exact Booth Multiplier." In *2025 IEEE 5th International Conference on VLSI Systems, Architecture, Technology and Applications (VLSI SATA)*, pp. 1–5. IEEE, 2025.